Assignment 1

Creation of Vehicle-Related Classes

Create the hierarchy of Vehicle interface and Boat and Car classes, as designed in the UML Class Diagram. All classes, enums and interfaces must be in edu.monmouth.vehicle package. Implement the following:

- Constructor must delegate setting of attributes to mutator methods and must throw a CarException or BoatException when a mutator generates an exception.
- engineSize mutator method must contain a precondition and generate a CarException (specifying the problem) if any of the preconditions fail:
  - engineSize is less than 1 or greater than 199
- boatLength mutator method must contain a precondition and generate a BoatException (specifying the problem) if any of the preconditions fail:
  - boatLength is less than 1 or greater than 1300
- Establish a constants file named VehicleConstants which contains only `public static final attributes`. This will contain all constants used by all classes implementing the Vehicle interface. It must supply a private default constructor preventing it from being instantiated. For example:

```
public static final int MINLENGTH = 0;
```

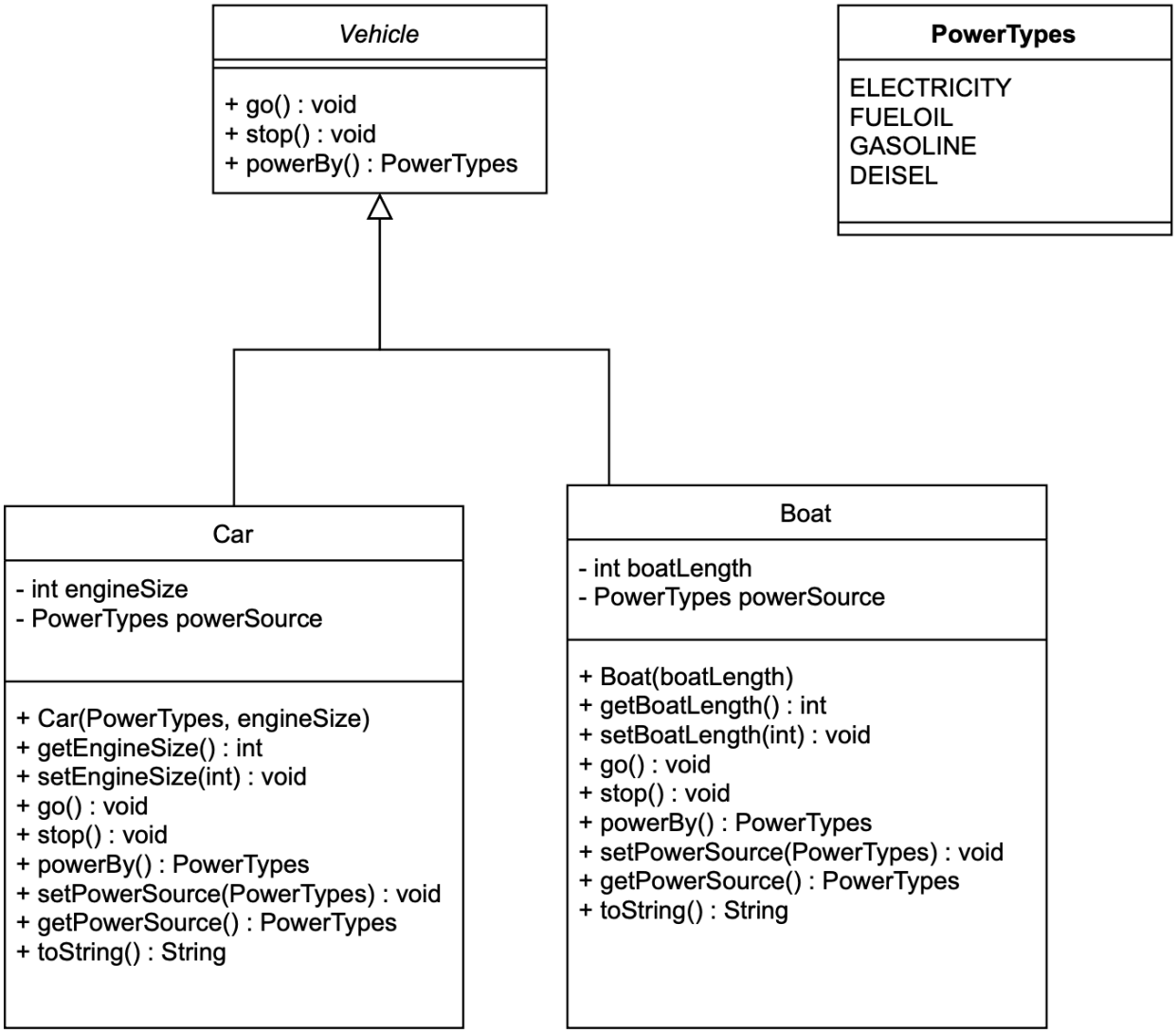
- Make use of constants for all preconditions (such as minimum and maximum engine size)
- Make use of the @Override annotation for all overridden methods

Create an enum named PowerTypes which specify values ELECTRICITY, GASOLINE, DEISEL, FUELOIL.

Create classes named CarException and BoatException which are derived from java.lang.Exception and will override the two Exception constructors, as described in class.

Create a linked list of Vehicle objects by:

- Creating a VehicleNode class as described in the UML Class Diagram
- Creating a VehicleList class as described in the UML Class Diagram
- Using the supplied HW1 class, instantiate a VehicleList, create Car and Boat objects and exercise VehicleList methods.


Vehicle
+ go() : void
+ stop() : void
+ powerBy() : PowerTypes
PowerTypes
ELECTRICITY
FUELOIL
GASOLINE
DEISEL
Car
- int engineSize
- PowerTypes powerSource
+ Car(PowerTypes, engineSize)
+ getEngineSize() : int
+ setEngineSize(int) : void
+ go() : void
+ stop() : void
+ powerBy() : PowerTypes
+ setPowerSource(PowerTypes) : void
+ getPowerSource() : PowerTypes
+ toString() : String
Boat
- int boatLength
- PowerTypes powerSource
+ Boat(boatLength)
+ getBoatLength() : int
+ setBoatLength(int) : void
+ go() : void
+ stop() : void
+ powerBy() : PowerTypes
+ setPowerSource(PowerTypes) : void
+ getPowerSource() : PowerTypes
+ toString() : String

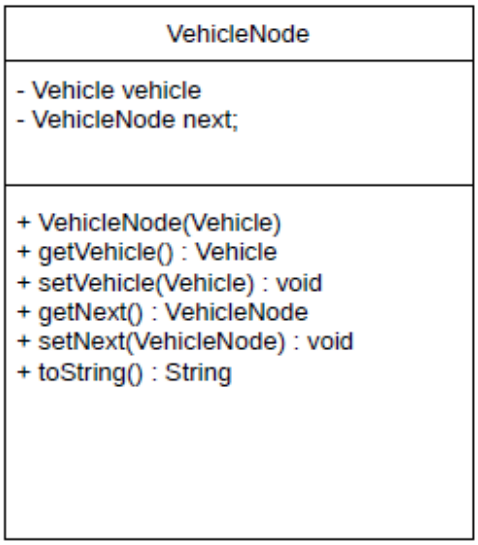
Linked List Classes

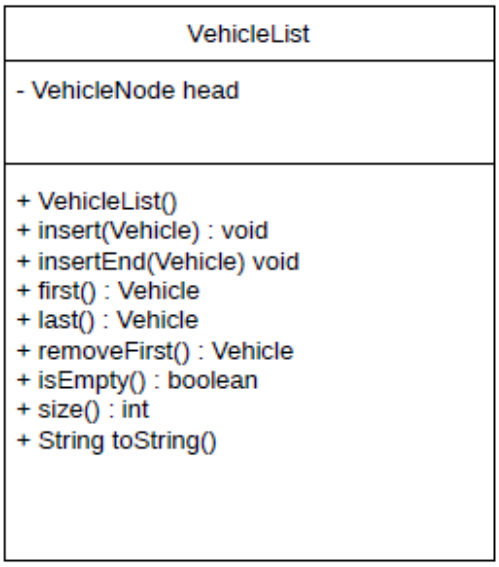
Download HW1.java from eCampus. Add HW1.java to the project's src directory. HW1 will be used to validate all other classes. Keep class HW1 in package edu.monmouth.hw1.

Create classes VehicleNode and VehicleListNode as described below. Classes HW1, VehicleNode and VehicleListNode are in package edu.monmouth.hw1. This will require import edu.monmouth.vehicle statements in these classes to access interface Vehicle, enum PowerTypes, classes Car, Boat and VehicleConstants. Keep these classes in their respective packages.

1. Give a body to the following VehicleList methods:
   - public void insert(Vehicle) – inserts the Vehicle object in the list.
   - public Vehicle first() – returns the Vehicle in the first VehicleNode in the list. That is, the VehicleNode to which `head` refers. Returns null when the list is empty.
   - public Vehicle last () – returns the Vehicle in the last VehicleNode in the list. That is, the VehicleNode which has a null `next` value. Returns null when the list is empty.
   - public Vehicle removeFirst() – removes and returns the Vehicle in the first VehicleNode in the list. Returns null when the list is empty.
   - public void insertEnd(Vehicle) – inserts the VehicleNode at the end of the list.
   - public boolean isEmpty() – returns true if there are no VehicleNodes in the list. Returns false otherwise.
   - public int size() – returns the number of VehicleNodes in the list.
   - public String toString() – returns a nicely formatted, easy to read String containing all attributes of each VehicleItem in the List.

Do not make any changes to HW1.java.





Assignment Questions

1) What is meant by self-referential class? Give a simple example.
2) Your assignment is to create a program that houses passengers on a boat. Boats will range from personal boats (just 1 or 2 passengers) to cruise ships (hundreds or thousands of passengers). Would you use an array or a linked list? Explain your answer.
3) The passengers are stored in an unordered list. You are given a passenger to find in your array or list. What is the Big O of this algorithm? Explain your answer.
4) Constructors have no return type. What is one method to inform a calling method that a runtime anomaly occurred in a constructor?
5) What is the Big O of your size() method? Explain your answer.
6) What is the Big O of your removeFirst() method? Explain your answer.
7) Why is it possible to return null when the return type of a method is Vehicle? For example, methods first() and last() have a return type of Vehicle, yet may return null if the list is empty.

Submit answers to assignment questions to eCampus in a file named Assignment1.txt.
Submit output proving all requirements are met to eCampus in a file named HW1output.txt (make this the value of the single command line argument).
Submit a link to your GitHub repo to eCampus

Submit all source INCLUDING ALL Vehicle-related files to GitHub.